FlashCP: Load-Balanced Communication-Efficient Context Parallelism for LLM Training

Zheng Wang [1] **Eric Liu** [2] **Linan Jiang** [1] **Zhongkai Yu** [1] **Zaifeng Pan** [1] **Yue Guan** [1] **Yuke Wang** [3] **Yufei Ding** [1]

Abstract

Context parallelism (CP) is essential for training large-scale, long-context language models, as it partitions sequences to reduce memory overhead. However, existing CP methods suffer from workload imbalance, inefficient kernels, and redundant communication due to static sequence sharding and key-value (KV) tensor communication. We present *FlashCP*, a load-balanced and communication-efficient framework for CP training. *FlashCP* introduces a sharding-aware communication mechanism to eliminate redundant KV communication and proposes a novel Whole-Doc sharding strategy that maximizes communication savings while maintaining balanced workloads. To efficiently combine Whole-Doc and Per-Doc sharding, *FlashCP* further designs a heuristic algorithm to search for near-optimal sharding plans. Extensive experiments show that *FlashCP* achieves up to 1.63× speedup over state-of-the-art CP frameworks across diverse datasets.

1. Introduction

Large language model (LLM) has demonstrate impressive capability in many tasks like translation (Zhang et al., 2023), reasoning (Guo et al., 2025), and coding (Wei et al., 2023; Nijkamp et al., 2022). The remarkable potential of LLMs has driven a growing trend among leading technology companies to developing increasingly larger-scale models with extended context windows, continually pushing the boundaries of LLM capabilities (Achiam et al., 2023; Grok, 3; Dubey et al., 2024; Team et al., 2023). For example, the context window size of the Llama model series evolves from 4K tokens in Llama2 (Touvron et al., 2023), to 128K tokens in Llama3 (Dubey et al., 2024), and finally reaching 10 million tokens in Llama4 (Singh, 2025).

[1]University of California San Diego, La Jolla, USA [2]University of Southern California, Los Angeles, USA [3]Rice University, Houston, USA. Correspondence to: Zheng Wang <zhw100@ucsd.edu>.

Preprint. June 9, 2026.

This increase in context window size introduces significant challenges for LLM training, primarily due to the growth in the activation size, which scales proportionally with the sequence length (Korthikanti et al., 2023). To address this challenge, in addition to the traditional 3D parallelism techniques (data parallelism, pipeline parallelism, and tensor parallelism) (Shoeybi et al., 2019; Narayanan et al., 2021; Team & Majumder, 2020), a new dimension called Context Parallelism (CP) has been introduced (NVIDIA, 2023). With CP, the inputs and activations are partitioned along the sequence length dimension and distributed across CP workers, allowing attention computation to run in parallel on different devices. This approach effectively reduces the memory consumption of attention layer activations on each device, offering a promising solution to train LLMs with large context windows.

Although context parallelism is essential for efficient long-context LLM training, achieving optimal performance remains challenging, and all existing solutions fall short in some aspects. **First**, balancing the attention workload across CP workers is difficult because the per-token attention computation varies with sequence position, leading to imbalanced workload distribution across GPUs. **Second**, CP introduces significant communication overhead due to the partitioning and distribution of the input sequence. Each token's attention depends on all preceding tokens, requiring the communication of KV tensors across CP workers. **Third**, maintaining high computation efficiency is nontrivial. Efficient attention kernels such as FlashAttention rely on sufficiently large query and KV lengths to achieve high GPU utilization, which is harder to sustain when the input is partitioned. Addressing any one of these challenges in isolation is relatively straightforward. However, simultaneously resolving all of them to achieve optimal performance is difficult due to the inherent trade-offs between these aspects. For example, a more fine-grained partitioning can improve workload balance but shortens each sequence shard, decreasing kernel efficiency (Wang et al., 2025b). Similarly, communication overhead can be mitigated by overlapping computation and communication, but this approach requires splitting the attention kernel into several smaller kernels to compute partial results and introduces additional result-processing overhead (Liu et al., 2023). Table 1 summarizes

Table 1. Comparison of CP approaches.

Method	Balance	Communication	Kernel Eff.
Llama3 CP (Per-Seq)	Imbalanced	High	High
Per-Doc CP	Balanced	High	Low
Ring-Attn (Zigzag)	Balanced	Moderate	Low
FlashCP	**Balanced**	**Low**	**High**

a comparison of mainstream CP approaches, all existing methods exhibit limitations in at least one dimension.

To address these challenges, we introduce *FlashCP*, a load-balanced and communication-efficient context parallelism framework for large-scale, long-context LLM training. *FlashCP* holistically optimizes the sharding strategy and the communication flow in CP, effectively balancing workload distribution, maximizing attention kernel efficiency, and minimizing communication overhead to achieve near-optimal CP training performance. Our key insight is that, rather than uniformly sharding and distributing all input documents, it is better to distribute the whole input document without sharding. Keeping the input document as a whole maximizes kernel efficiency and eliminates the need to communicate the KV tensor across GPUs. To achieve balanced attention workload distribution with minimal document sharding, *FlashCP* incorporates several key optimizations: **First**, to minimize the communication overhead, *FlashCP* employs a sharding-aware communication mechanism, communicating only the necessary portions of KV tensors required by each CP worker, thereby eliminating redundant data transfers. **Second**, *FlashCP* proposes a novel *Whole-Doc* sharding strategy, which keeps short documents as a whole on a single CP worker to reduce the required communication amount and adaptively shards the remaining documents to achieve balanced workload distribution and high attention kernel efficiency. **Third**, *FlashCP* introduces a heuristic sharding algorithm that overcomes the NP-hardness of the sharding problem and efficiently searches for the near-optimal sharding plan that combines both *Whole-Doc* and *Per-Doc* sharding strategies.

In summary, this paper makes the following contributions:

- We reveal the limitations of existing CP frameworks, which fail to maintain high kernel efficiency and suffer from redundant KV tensor communication.
- We propose *FlashCP*, which holistically optimizes input sharding and communication flow to achieve balanced workloads, reduced communication, and high kernel efficiency.
- We compare *FlashCP* with state-of-the-art CP framework and observe up to $1.63\times$ speedup across various datasets.

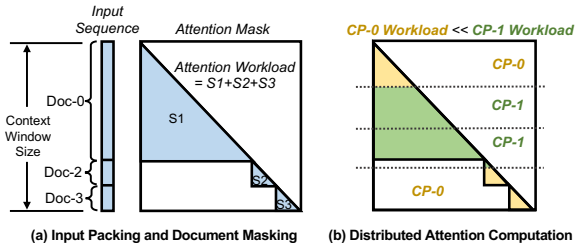


Figure 1. Illustration of input packing and distributed attention computation in context parallelism.

2. Background and Motivation

2.1. Input Packing and Document Mask

Input documents in LLM training exhibit highly skewed length distributions (An et al., 2024; Jiang et al., 2024; Wang et al., 2025b). Early approaches rely on zero-padding to align sequences within a batch (Shoeybi et al., 2019). However, this padding approach introduces redundant computation, communication, and memory overhead. To address this, input packing was proposed to concatenate multiple short documents into a single long sequence (Zhao et al., 2024; Raffel et al., 2020; Krell et al., 2021; Wang et al., 2024). Built upon input packing, the *Document Mask* (also known as intra-document causal masking) was proposed to mask out cross-document attention computation and ensure correct attention behavior (Pytorch, 2024; NVIDIA; Zhao et al., 2024; Kundu et al., 2024). Figure 1(a) shows an example of input packing and document masking. The combination of input packing and document masking has emerged as a widely adopted paradigm for large-scale, long-context LLM training (Wang et al., 2025b; Ge et al., 2025) and has been successfully applied in industry-scale models such as Llama3 (Dubey et al., 2024).

2.2. Context Parallelism

Context parallelism (CP) mitigates the large activation memory induced by long context windows by partitioning input sequences along the sequence-length dimension across multiple workers (NVIDIA, 2023; Gu et al., 2024; Dubey et al., 2024; Wang et al., 2025b). Each worker processes a subset of tokens and computes attention locally, while exchanging KV tensors to obtain the full attention context. Figure 1(b) shows an example following the Llama3 CP implementation (Dubey et al., 2024), where the input sequence is split into $2 \times$ CP_size shards, and each worker processes two shards. This static sharding strategy can lead to significant workload imbalance across workers, highlighting the need for optimized document sharding and token distribution.

2.3. Limitation of Existing Works

The Llama3 CP (Dubey et al., 2024) adopts a coarse-grained input sharding strategy that uniformly splits the

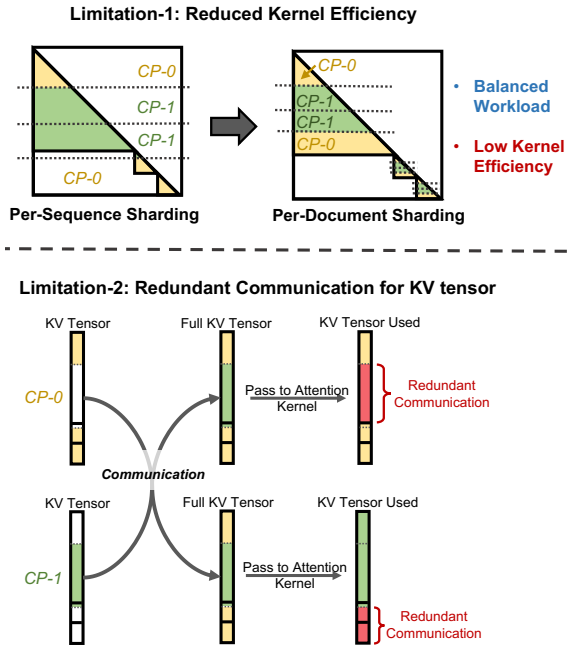


Figure 2. Limitations of existing CP training frameworks: (1) Existing CP implementations employ fine-grained per-document sharding to achieve balanced workload distribution, which reduces attention kernel efficiency; (2) Existing works communicate the full KV tensors across all CP workers, leading to redundant KV tensor transfers and unnecessary communication overhead.

entire input sequence. This approach often leads to severe workload imbalance across CP workers. To address this issue, two more advanced frameworks have been proposed: *Per-Doc CP* (Wang et al., 2025b) and *Ring-Attn (Zigzag)* (Zhuzilin, 2025). Both frameworks employ a fine-grained, per-document sharding strategy, where each input document is divided into $2 \times$ CP_size chunks. Chunks i and $(2N - 1 - i)$ are then assigned to the i-th CP worker. As shown in Figure 2, this fine-grained document sharding strategy could achieve balanced workload distribution across CP workers. Although both methods effectively achieve workload balance, they share several common limitations:

Reduced Kernel Efficiency: Fine-grained sharding causes each attention kernel to operate on shorter document shards, which decreases kernel efficiency (Wang et al., 2025b). To demonstrate this effect, we profiled kernel execution latency using two input patterns: one with a single document of length 128K, and another with 16×8K documents. As shown in Figure 3, *Per-Doc* sharding incurs noticeably higher attention latency, particularly for workloads dominated by short documents. *Ring-Attn (Zigzag)* shows even higher latency since it computes attention block-by-block, introducing additional partial result aggregation overhead.

Redundant Communication: Although the two frameworks adopt different communication approaches, they both suffer from redundant communication. *Per-Doc CP* relies on collective communication (e.g., `AllGather` and `ReduceScatter`), while *Ring-Attn (Zigzag)* uses peer-to-peer (P2P) communication. In both cases, the entire KV tensors are transferred across all CP workers. However, each worker only requires a subset of the KV tensors to compute attention, resulting in unnecessary data transfer, as illustrated in Figure 2.

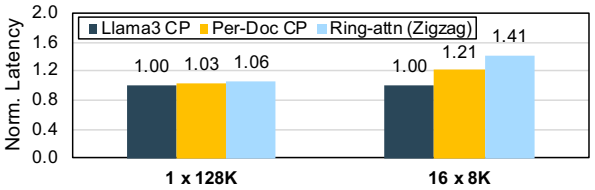


Figure 3. Kernel efficiency comparison: Fine-grained per-document sharding decreases attention kernel efficiency.

3. FlashCP Design

3.1. Optimization Goal and Problem Formulation

Our goal is to determine an input sharding and distribution strategy that balances computation across CP workers while minimizing KV communication overhead and kernel efficiency degradation. We consider context parallelism with CP size N and context window C. Given an input sequence comprising n documents $\mathbf{D} = [d_1, d_2, \cdots, d_n]$, where d_i denotes the length of the i-th document, the documents are further partitioned into m document shards $\mathbf{S} = [s_1, s_2, \cdots, s_m]$, where s_i is the shard length. Each shard is also associated with a prefix length p_i, representing the number of tokens preceding its starting position.

Input Shard Distribution. For an input document shard s_i, its distribution is represented by an array of binary variables: $\mathbf{x}_i = [x_{i1}, x_{i2}, \cdots, x_{iN}]$, where $x_{ij} = 1$ if the shard s_i is assigned to the j-th CP worker. Each shard is assigned to exactly one worker, enforced by the constraint:

$$\sum_{j=1}^{N} x_{ij} = 1, \quad \forall i \in \{1, 2, \cdots, m\} \tag{1}$$

Equal Token Constraint. To balance computation across CP workers, each worker must process the same number of tokens, since the cost of non-attention layers (e.g., FFN) scales linearly with token count. This requirement is enforced by the constraint:

$$\sum_{i=1}^{m} (x_{ij} \cdot s_i) = \frac{C}{N}, \quad \forall j \in \{1, 2, \cdots, N\} \tag{2}$$

Balancing Computation Workload. Since training operates in a synchronized manner, the overall performance is bottlenecked by the slowest CP worker. Therefore, the

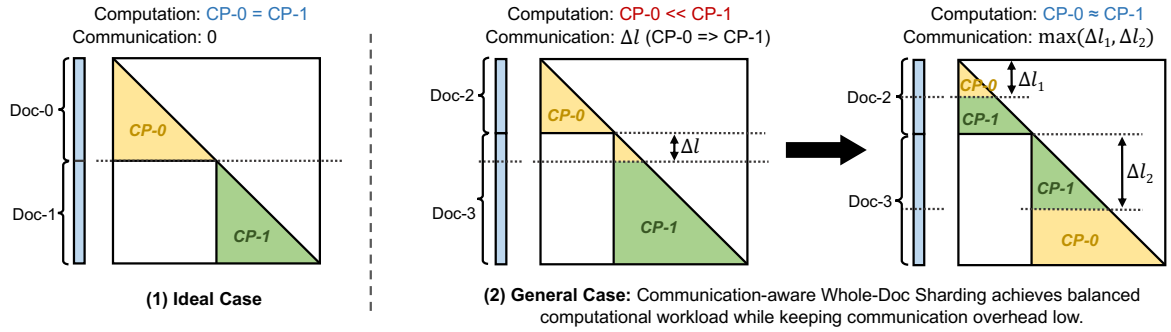


Figure 4. Illustration of communication-aware *Whole-Doc* sharding.

objective is to minimize the maximum attention computation workload across all workers, subject to the constraints defined in Eq. 1 and Eq. 2:

$$\underset{\mathbf{X},\mathbf{S}}{\text{Minimize}} \left(\max_{j \in 1,2,\cdots,N} \sum_{i=1}^{m} (x_{ij} \cdot W_i) \right) \tag{3}$$

Here, W_i is the attention computational workload of the i-th input shard, which is computed as $W_i = (2 \cdot p_i + s_i + 1) \times s_i/2$.

3.2. Dynamic Sharding-aware Communication

A missing component of the problem formulation in Section 3.1 is the communication overhead. Existing CP implementations employ a **static communication pattern** that transfers the full KV tensors across all CP workers. The total communication volume along the critical path can be expressed as:

$$4 \times \frac{\sum_{i=1}^{n} d_i}{N} \times H \times D \times (N-1) \tag{4}$$

Here, H denotes the number of attention heads, D the head dimension, and N the number of CP workers. The factor of $4\times$ arises because communication is required for both the K and V tensors in both the forward and backward passes. Since CP is typically applied across nodes, where communication bandwidth is significantly lower than that of intra-node NVLink, the communication overhead could become a significant bottleneck.

This approach is inefficient as it introduces redundant communication. To mitigate communication overhead in CP training, we propose a **dynamic sharding-aware communication** strategy, which adaptively determines the size of the communication buffer based on the sharding plan, thereby avoiding redundant KV tensor transfers. Specifically, if an input document is assigned to a single CP worker, it is skipped for communication, since the CP worker already holds the full KV tensor for this document and can perform the attention computation locally. For input documents that are sharded and distributed to multiple CP workers, the communication size is proportional to the maximum prefix length among all shards. This is because the Q tensor of each token only computes attention with the prefix part of the KV tensor in an input document. Moreover, since each document shard may have a different prefix length, this results in zero padding in the communication buffer for each input document. To avoid this, rather than handling each document individually, we use a single continuous communication buffer and compact the KV tensors corresponding to the prefix portions of all document shards into the buffer. With our dynamic sharding-aware strategy, the communication size is reduced to:

$$4 \times \left(\max_{j \in \{1,2,\cdots,N\}} \sum_{i \in \hat{\mathbf{S}}} (x_{ij} \cdot s_i) \right) \times H \times D \times (N-1) \tag{5}$$

Here, $\hat{\mathbf{S}}$ denotes the set of input document shards excluding the last shard of each input document. An entire document that is not further divided is also treated as the last shard and, therefore, is not included in $\hat{\mathbf{S}}$. By applying the dynamic sharding-aware communication strategy, communication is reduced for two types of input shards: (1) documents that are fully assigned to a single CP worker, and (2) the last shard of each input document, achieving significant communication savings.

3.3. Whole-Doc Sharding for Communication Savings

To maximize communication savings, we ideally want to each input document intact on a single CP worker, eliminating KV communication and preserving kernel efficiency by enabling single-kernel attention computation. As shown in Figure 4(1), in the ideal case of two equal-length documents, the optimal strategy is to keep each document intact and assign it to a separate CP worker, achieving balanced computation and eliminating communication since each worker holds the full KV tensor. However, in practice, documents have varying lengths, making it difficult to preserve whole documents while evenly distributing workloads across CP

workers. In Figure 4 (2), when the input contains documents of different lengths, the longer document must be split to satisfy the equal token constraint in Eq. 2. While this sharding plan incurs relatively low communication overhead (proportional to Δl), it introduces significant workload imbalance. To address this challenge, we propose a **Communication-aware Whole-Doc Sharding** method, which adaptively shards documents to balance workload while minimizing communication overhead. Specifically, instead of naively partitioning only the longest document to achieve equal token distribution across CP workers, we propose adaptively partitioning multiple documents. As shown in the right part of Figure 4 (2), both *Doc-2* and *Doc-3* are partially partitioned to achieve balanced workloads and equal token counts per worker. Moreover, since only the lower segments of the documents need to be exchanged, the communication volume is bounded by $\max(\Delta l_1, \Delta l_2)$, significantly reducing overall communication overhead. This method is referred to as *Whole-Doc Sharding*, as it aims to preserve each document as a whole and only applies adaptive sharding when necessary to balance the workload distribution.

Combine *Per-Doc* and *Whole-Doc* Sharding. Although *Whole-Doc* sharding reduces communication overhead and can help achieve workload balance across CP workers, it cannot efficiently handle all combinations of input documents. In certain cases, for example, when the input sequence contains an extremely long document that contributes the majority of tokens, *Whole-Doc* sharding fails to achieve balanced workload distribution due to the significant disparity between the long document and the remaining short documents. To address this, we design a hybrid sharding strategy that combines both *Per-Doc* and *Whole-Doc* sharding. Specifically, we select the appropriate sharding method for each document based on its length. For extremely long documents that cause workload imbalance, *Per-Doc* sharding is applied to evenly distribute computation, while maintaining high attention kernel efficiency due to sufficient token counts. In contrast, *Whole-Doc* sharding is used for shorter documents to reduce communication overhead and achieve balanced workloads when document length disparity is moderate.

3.4. FlashCP Sharding Algorithm

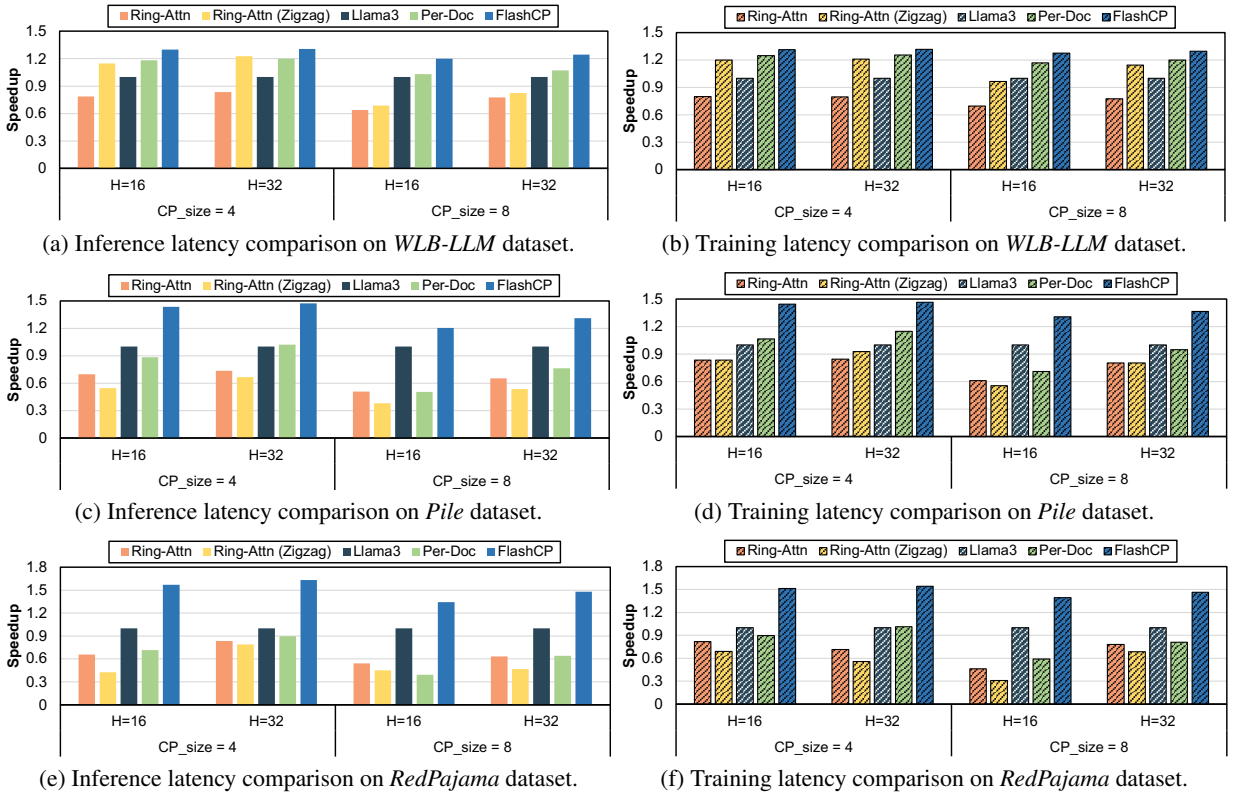
ILP-based Sharding: Based on the problem formulation in Section 3.1, we can model the sharding task as an integer linear programming (ILP) problem by incorporating the communication overhead term (Eq. 5) into the objective function. This formulation allows us to employ ILP solvers to obtain an optimal input sharding and distribution plan for a given sharding granularity. However, the computational cost of solving the ILP is prohibitively high for practical use, which motivates the design of a more efficient heuristic sharding algorithm to search for a near-optimal solution.

Algorithm 1 Heuristic sharding algorithm of FlashCP.

Input: inputs $\mathbf{D} = [d_1, d_2, \cdots, d_n]$, Target Imbalance Ratio R
Output: Sharding Plan Per_Doc_P and $Whole\_Doc\_P$

1: Sort $\mathbf{D}$ in descending order of their lengths.
2: Initialize empty per-doc sharding plan Per_Doc_P
3: Initialize current imbalance ratio $Cur\_R = Inf$
4: **while** $Cur\_R > R$ **do**
5: Initialize empty temporary whole-doc sharding plan tmp_P
6: **for** doc d in $\mathbf{D}$ **do**
7: // Add d to the least-loaded worker:
8: $tmp\_P.Min\_Worker\_Add(d)$
9: **end for**
10: // Make sure Eq. 2 is satisfied:
11: **while** tmp_P is not equal token **do**
12: // Pop documents from the worker with the most tokens:
13: $docs = tmp\_P.Max\_Token\_Worker\_Pop()$
14: // Apply Whole-Doc sharding:
15: $tmp\_P.Whole\_Doc\_Shard\_and\_Add(docs)$
16: **end while**
17: // Update current imbalance ratio:
18: $Cur\_R = T.Compute\_Imba\_Ratio(tmp\_p)$
19: **if** $Cur\_R > R$ **then**
20: // Pop the longest document and apply Per-Doc sharding:
21: $d = \mathbf{D}.Pop\_Front()$
22: $Per\_Doc\_P.Add(d)$
23: **end if**
24: **end while**
25: $Whole\_Doc\_P = tmp\_P$
26: **Return** Per_Doc_P and $Whole\_Doc\_P$

FlashCP Heuristic Sharding Algorithm: To efficiently search for a near-optimal sharding plan, we propose a greedy heuristic sharding algorithm. The details of the algorithm are given in Algorithm 1. The algorithm takes the document sequence $\mathbf{D} = [d_1, d_2, \cdots, d_n]$ and the target imbalance ratio $R = \frac{max\_workload}{avg\_workload}$ as input. Here, $max\_workload$ and $avg\_workload$ are the maximum and average attention computation workloads across all CP workers, respectively. The algorithm first sorts documents by decreasing length, then iteratively constructs a temporary plan tmp_p by assigning each document to the worker with the minimum workload (lines 5–9). During this process, the algorithm distributes entire documents to specific CP workers without sharding, which helps maximize communication savings and maintain kernel efficiency. After that, the algorithm checks the number of tokens assigned to each worker. If the token counts are not equal, it applies *Whole-Doc* sharding to balance both the token distribution and the attention computation workload (lines 10–16). After that, tmp_p becomes a valid sharding plan that satisfies the equal-token constraint (Eq. 2), and the algorithm computes the imbalance ratio Cur_R of the current sharding plan tmp_p (line 18). If the imbalance ratio of tmp_p is larger than R, it indicates that there may be some extremely long sequences that hinder load balancing. To address this issue, the algorithm removes



(a) Inference latency comparison on *WLB-LLM* dataset.

(b) Training latency comparison on *WLB-LLM* dataset.

(c) Inference latency comparison on *Pile* dataset.

(d) Training latency comparison on *Pile* dataset.

(e) Inference latency comparison on *RedPajama* dataset.

(f) Training latency comparison on *RedPajama* dataset.

Figure 5. CP training and inference performance comparison: (a)(b) are on *WLB-LLM*, (c)(d) are on *Pile*, and (e)(f) are on *RedPajama*. H refers to the number of attention heads. The context window size is set to 128K and the head dimension is 128. The speedup data are normalized to the latency of *Llama3 CP* baseline.

the longest document from the input sequence and applies *Per-Doc* sharding to it (lines 19–23). The remaining documents are then used in the subsequent iterations. This process is repeated until the achieved imbalance ratio Cur_R becomes less than the target ratio R. The algorithm then returns tmp_p as the final *Whole-Doc* sharding plan, along with the set of documents assigned to *Per-Doc* sharding.

4. Experiments

4.1. Experiment Setup

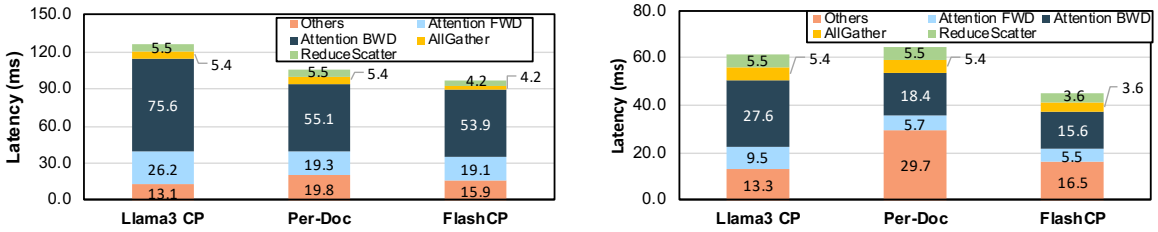
Experiment Environments: All of our experiments are conducted on a single node with 8× NVIDIA H100 SXM 80GB GPUs interconnected via high-bandwidth NVLink.

Baselines: We compare *FlashCP* against several state-of-the-art CP training frameworks:

- *Ring-Attn* (Liu et al., 2023): The pioneering CP method using P2P communication to exchange KV and overlap computation and communication. As the original version does not support input packing, we adopt an improved open-source version (Zhuzilin, 2025).
- *Ring-Attn (Zigzag)* (Zhuzilin, 2025): An enhanced variant that uses fine-grained sharding to improve workload balance while retaining ring-based communication.
- *Llama3 CP* (Dubey et al., 2024): The CP framework used in Llama3, which uniformly shards the whole input sequences and employs collective communication primitives (`AllGather`/`ReduceScatter`) for KV tensor and gradient synchronization.
- *Per-Doc CP* (Wang et al., 2025b): A advanced version of *Llama3 CP* that adopts per-document sharding to achieve better load balance and end-to-end efficiency, while using a similar communication pattern.

Datasets. We evaluate *FlashCP* on three LLM datasets:

- *WLB-LLM* (Wang et al., 2025b): We use the document length distribution released in the *WLB-LLM* paper and randomly generate documents that follow this distribution. The data distribution was collected from production-level training data used by Meta.
- *Pile* (Gao et al., 2020): The Pile is a large-scale English text dataset designed for training large language models. It has been widely used in many open-source LLM training (Black et al., 2021; Touvron et al., 2023).



(a) Latency breakdown on *WLB-LLM* dataset.

(b) Latency breakdown on *Pile* dataset.

Figure 6. The training latency breakdown of *FlashCP* and two baselines on the *WLB-LLM* and *Pile* datasets. The data are collected from the intra-node experiments with 8 CP workers.

- *RedPajama* (Computer, 2023): RedPajama is built by Together.ai and is an open-source recipe for reproducing the LLaMA training dataset. It consists of 1.2 trillion tokens drawn from diverse sources such as CommonCrawl, C4, GitHub, arXiv, and more.

Due to the large size of each dataset, we randomly sample 100K input sequences from each for evaluation. Each input sequence is composed of multiple documents. If the total length of the input documents exceeds the context window size, the last document is truncated to fit within the limit.

4.2. Training and Inference Performance Comparison

We evaluate the CP training and inference performance of *FlashCP* and all baselines on three benchmark datasets, using two model configurations with 16 and 32 attention heads (head dimension 128) and a context window of 128K. We experiment with CP sizes of 4 and 8.

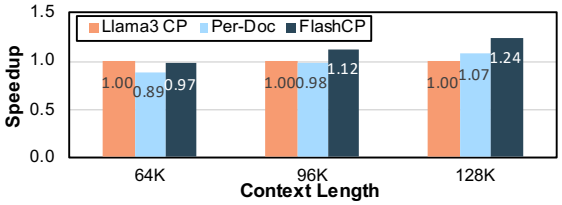
Comparison to *Llama3 CP* and *Per-Doc CP*: As shown in Figure 5, *FlashCP* consistently outperforms both baselines across various configurations, achieving average speedups of $1.38\times$ over *Llama3 CP* and $1.63\times$ over *Per-Doc CP*. Compared to *Llama3 CP*, the performance gain mainly arises from improved workload balance. *Llama3 CP* uniformly shards the input sequence, leading to load imbalance. In contrast, *FlashCP* adopts a hybrid, adaptive sharding strategy that distributes input documents in a workload-aware manner, achieving significantly better load balancing. Compared to *Per-Doc CP*, the improvement stems from reduced communication overhead and higher kernel efficiency. Although *Per-Doc CP* achieves load balance via fine-grained sharding, it incurs extra overhead from inefficient attention kernels and frequent small data transfers. *FlashCP* addresses this by applying a *Whole-Doc* sharding strategy that maintains good workload balance while avoiding unnecessary sharding of short input documents. Additionally, its dynamic communication optimization further reduces communication overhead, significantly improving the overall performance.

Comparison to *Ring-Attn* and *Ring-Attn (Zigzag)*: *FlashCP* achieves an average speedup of $1.97\times$ over the *Ring-Attn* baseline and $2.14\times$ over the *Ring-Attn (Zigzag)* baseline. Although both *Ring-Attn* and *Ring-Attn (Zigzag)* overlap communication with computation to mitigate communication overhead, they still suffer from low kernel efficiency. This inefficiency arises from two main factors. First, the ring-based attention mechanism requires splitting the original attention kernel into multiple smaller blockwise kernels, which reduces overall kernel utilization. Second, since each kernel computes only a partial attention result, both methods require an additional partial result processing steps to obtain the final attention output.

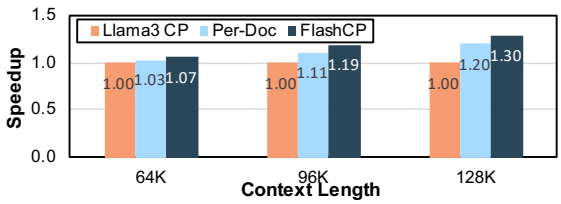
Across different datasets: *Per-Doc CP* and *Ring-Attn (Zigzag)* perform well on *WLB-LLM* but degrade significantly on *Pile* and *RedPajama*. The reason is the difference in document length distribution across dataset. *WLB-LLM* is more skewed with extremely long documents, while *Pile* and *RedPajama* contain more shorter sequences. Both *Per-Doc CP* and *Ring-Attn (Zigzag)* employs fine-grained per-document sharding strategy, which will reduce the input shard length and decrease the kernel efficiency especially when the input sequence is mostly consists of multiple short documents. In contrast, *FlashCP* achieves consistently strong performance across all datasets, demonstrating the robustness of its adaptive sharding strategy.

4.3. Optimization Analysis

Training Latency Breakdown: To analyze the performance gains of *FlashCP* and the impact of each individual optimization, we present a training latency breakdown of *FlashCP* and two baselines on *WLB-LLM* and *Pile*. Results are shown in Figure 6. For communication latency (AllGather and ReduceScatter), *Llama3 CP* and *Per-Doc CP* incur similar costs due to full KV tensor exchange. In contrast, *FlashCP* only communicates the required portions of the KV tensor for each CP worker, reducing latency by 23.6% and 34.5% on *WLB-LLM* and *Pile*, respectively. Regarding attention kernel latency, both *Per-Doc CP* and *FlashCP* outperform *Llama3 CP* due to improved workload balance. Moreover, *FlashCP* further reduces latency over *Per-Doc CP* by avoiding unnecessary fine-grained sharding, thereby preserving higher kernel efficiency. The remaining portion



(a) CP inference speedup.



(b) CP training speedup.

Figure 7. Speedups of *FlashCP* across different context window sizes. The data are collected from the intra-node experiments with 8 CP workers using the *WLB-LLM* dataset.

Table 2. Comparison between ILP Solver and Heuristic Algorithm.

Metric	ILP Solver	Heuristic
Communication Saving	36%	28%
Workload Imbalance Ratio	1.00	1.04

of the latency (labeled as *Others*) comes from data copy overhead. *Per-Doc CP* introduces significantly higher data copy latency, since its fine-grained sharding produces many small KV tensor copies. Instead, *FlashCP* introduces only modest overhead by selectively sharding long documents while keeping short ones intact.

Speedup Across Context Window Sizes: We investigate the impact of context window size on the performance gains delivered by *FlashCP*. We evaluate *FlashCP* and all baselines across context window sizes ranging from 64K to 128K. As shown in Figure 7, *FlashCP* outperforms both baselines across almost all context window size configurations. Moreover, as the context window size increases, the speedup achieved by *FlashCP* becomes more pronounced. This is because the attention computation cost increases quadratically with context length, causing larger context windows to exacerbate workload imbalance across CP workers. This further highlights the effectiveness of *FlashCP* in mitigating such imbalance. This increasing trend in speedup demonstrates the strong potential of *FlashCP* in long-context LLM training, especially given the ongoing trend toward ever-larger context window sizes.

Optimality Study: We evaluate the optimality of our heuristic sharding algorithm by comparing its communication saving and workload imbalance ratios against an ILP solver on the *Pile* dataset with 4 CP workers. The workload imbalance ratio is defined as $\frac{max\_workload}{avg\_workload}$, where *max_workload* and *avg_workload* denote the maximum and average attention workloads across all CP workers, respectively. As shown in Table 2, the ILP solver achieves a communication saving ratio of 36% and a workload imbalance ratio of 1.00. However, it incurs high computational cost. Solving a single input sequence can take tens of minutes, making it impractical for real-world use. In contrast, our heuristic achieves 28% communication saving and a 1.04 imbalance ratio, both very close to those of the ILP solver. These results demonstrate that the heuristic algorithm effectively balances communication savings and workload distribution, achieving near-optimal performance.

5. Related Works

As LLM context windows continue to grow (Team et al., 2023; 2024; Singh, 2025; Zhu et al., 2025), training becomes increasingly constrained by the rapidly growing activation memory of attention layers (Korthikanti et al., 2023; Wang et al., 2025a). CP has recently emerged as an effective strategy that partitions input sequences and activations along the sequence dimension, distributing attention computation across GPUs (NVIDIA, 2023; Gu et al., 2024; Dubey et al., 2024; Wang et al., 2025b). Early CP methods adopt ring-based communication to exchange KV tensors among workers (Liu et al., 2023), enabling partial overlap of computation and communication but struggling to support the complex attention masks required by input packing. To address this limitation, zigzag-style sharding is introduced (Zhuzilin, 2025). However, it still suffers from reduced kernel efficiency due to blockwise attention execution. More recent approaches leverage collective communication primitives, such as AllGather (NVIDIA, 2023; Dubey et al., 2024) and AlltoAll (Jacobs et al., 2023), to enable efficient attention computation with document masking by providing each worker with a global KV view. However, these methods communications the entire KV tensor, introducing significant redundant communication.

6. Conclusion

In this paper, we introduce *FlashCP*, a load-balanced and communication-efficient context parallelism framework for large-scale, long-context LLM training. Specifically, *FlashCP* proposes a sharding-aware communication mechanism that minimizes unnecessary communication, and a novel *Whole-Doc* sharding strategy to maximize communication savings while maintaining balanced workload distribution across CP workers. Moreover, *FlashCP* further introduces a heuristic sharding algorithm to efficiently search for a near-optimal sharding plan. Extensive experiments show that *FlashCP* delivers up to 1.63× speedup over state-of-the-art CP frameworks across diverse datasets.

References

Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023.

An, C., Zhang, J., Zhong, M., Li, L., Gong, S., Luo, Y., Xu, J., and Kong, L. Why does the effective context length of llms fall short? *arXiv preprint arXiv:2410.18745*, 2024.

Black, S., Leo, G., Wang, P., Leahy, C., and Biderman, S. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow, March 2021. URL https://doi.org/10.5281/zenodo.5297715. If you use this software, please cite it using these metadata.

Computer, T. Redpajama: An open source recipe to reproduce llama training dataset, 2023. URL https://github.com/togethercomputer/RedPajama-Data.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*, 2024.

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

Ge, H., Feng, J., Huang, Q., Fu, F., Nie, X., Zuo, L., Lin, H., Cui, B., and Liu, X. Bytescale: Efficient scaling of llm training with a 2048k context length on more than 12,000 gpus. *arXiv preprint arXiv:2502.21231*, 2025.

Grok, X. Beta—the age of reasoning agents— xai, 2025. *URL https://x.ai/news/grok-3*, 3.

Gu, D., Sun, P., Hu, Q., Huang, T., Chen, X., Xiong, Y., Wang, G., Chen, Q., Zhao, S., Fang, J., et al. Loongtrain: Efficient training of long-sequence llms with head-context parallelism. *arXiv preprint arXiv:2406.18485*, 2024.

Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Jacobs, S. A., Tanaka, M., Zhang, C., Zhang, M., Song, S. L., Rajbhandari, S., and He, Y. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. *arXiv preprint arXiv:2309.14509*, 2023.

Jiang, C., Jia, Z., Zheng, S., Wang, Y., and Wu, C. DynaPipe: Optimizing Multi-task Training through Dynamic Pipelines. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pp. 542–559, 2024.

Korthikanti, V. A., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoeybi, M., and Catanzaro, B. Reducing Activation Recomputation in Large Transformer Models. *Proceedings of Machine Learning and Systems*, 5: 341–353, 2023.

Krell, M. M., Kosec, M., Perez, S. P., and Fitzgibbon, A. Efficient Sequence Packing without Cross-contamination: Accelerating Large Language Models without Impacting Performance. *arXiv preprint arXiv:2107.02027*, 2021.

Kundu, A., Lee, R. D., Wynter, L., Ganti, R. K., and Mishra, M. Enhancing training efficiency using packing with flash attention. *arXiv preprint arXiv:2407.09105*, 2024.

Liu, H., Zaharia, M., and Abbeel, P. Ring Attention with Blockwise Transformers for Near-Infinite Context. *arXiv preprint arXiv:2310.01889*, 2023.

Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., et al. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15, 2021.

Nijkamp, E., Pang, B., Hayashi, H., Tu, L., Wang, H., Zhou, Y., Savarese, S., and Xiong, C. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. *arXiv preprint arXiv:2203.13474*, 2022.

NVIDIA. NVIDIA NeMo Framework: Sequence Packing. https://docs.nvidia.com/nemo-framework/user-guide/latest/sft_peft/packed_sequence.html.

NVIDIA. Megatron Core: Context Parallelism. https://docs.nvidia.com/megatron-core/developer-guide/latest/api-guide/context_parallel.html, 2023.

Pytorch. FlexAttention: The Flexibility of PyTorch with the Performance of FlashAttention. https://pytorch.org/blog/flexattention/, 2024.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21 (140):1–67, 2020.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Singh, A. Meta llama 4: The future of multimodal ai. *Available at SSRN 5208228*, 2025.

Team, D. and Majumder, R. DeepSpeed: Extreme-Scale Model Training for Everyone, 2020.

Team, G., Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

Team, G., Georgiev, P., Lei, V. I., Burnell, R., Bai, L., Gulati, A., Tanzer, G., Vincent, D., Pan, Z., Wang, S., et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*, 2023.

Wang, S., Wang, G., Wang, Y., Li, J., Hovy, E., and Guo, C. Packing analysis: Packing is more appropriate for large models or datasets in supervised fine-tuning. *arXiv preprint arXiv:2410.08081*, 2024.

Wang, T., Chen, X., Li, K., Cao, T., Ren, J., and Zhang, Y. Lemo: Enabling less token involvement for more context fine-tuning. *arXiv preprint arXiv:2501.09767*, 2025a.

Wang, Z., Cai, A., Xie, X., Pan, Z., Guan, Y., Chu, W., Wang, J., Li, S., Huang, J., Cai, C., et al. Wlb-llm: Workload-balanced 4d parallelism for large language model training. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025b.

Wei, Y., Xia, C. S., and Zhang, L. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 172–184, 2023.

Zhang, B., Haddow, B., and Birch, A. Prompting Large Language Model for Machine Translation: A Case Study. In *International Conference on Machine Learning*, pp. 41092–41110. PMLR, 2023.

Zhao, Y., Qu, Y., Staniszewski, K., Tworkowski, S., Liu, W., Miłoś, P., Wu, Y., and Minervini, P. Analysing the impact of sequence composition on language model pre-training. *arXiv preprint arXiv:2402.13991*, 2024.

Zhu, T., Liu, Q., Wang, H., Chen, S., Gu, X., Pang, T., and Kan, M.-Y. Skyladder: Better and faster pretraining via context window scheduling. *arXiv preprint arXiv:2503.15450*, 2025.

Zhuzilin. ring-flash-attention: Ring attention implementation with flashattention. https://github.com/zhuzilin/ring-flash-attention, 2025. URL https://github.com/zhuzilin/ring-flash-attention. GitHub repository.